5. Gyakorlat

Láncolt listák

Kétirányú lista

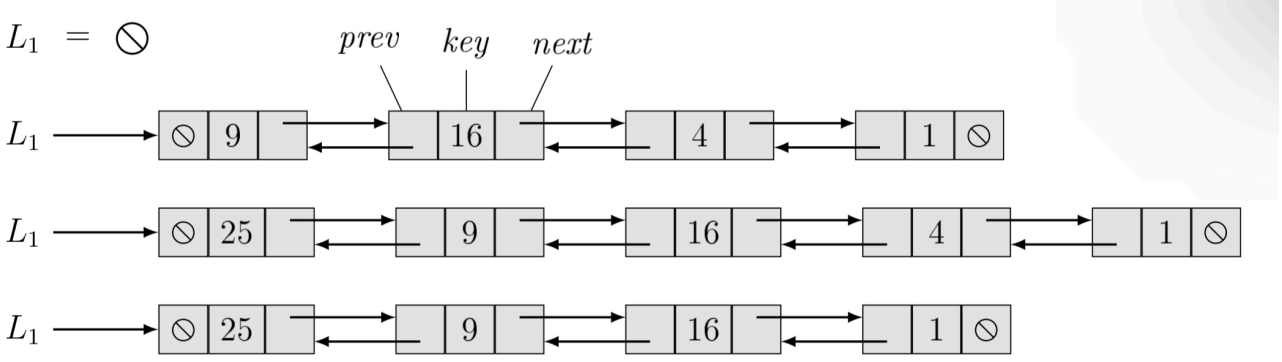
- *egyszerű kétirányú listák (S2L - Simple 2-way List)*
- *fejelemes egyirányú listák (C2L - Cyclic 2-way List with header)*

Listaelem típusa

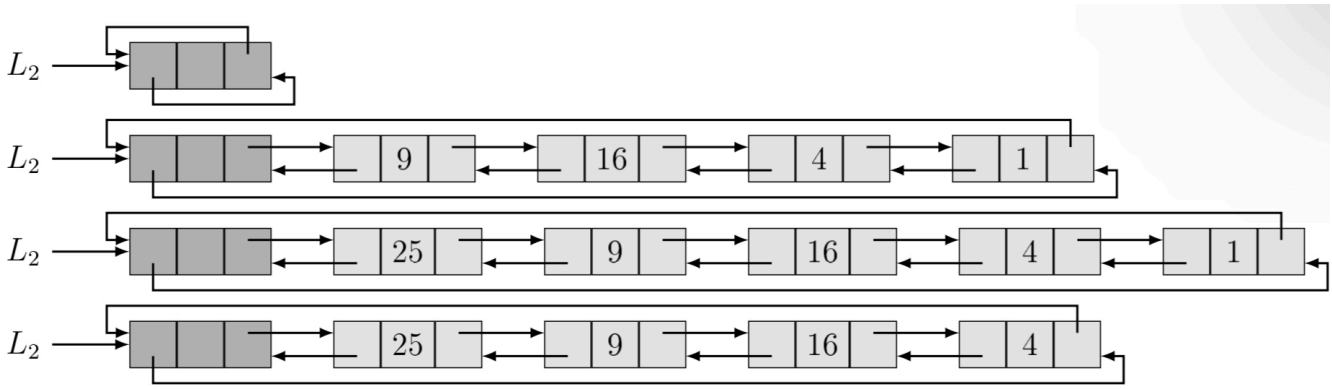
E2
$+prev, next$: **E2*** // refer to the previous and next neighbour or be **this** $+key : \mathcal{T}$
+ E2() { $prev := next :=$ **this** }

- a lista bejárásához pointereket használunk:
  - `p:E2*` pointer
    - a mutatott objektum adattagjai:
      - `p -› key`
      - `p -› next`
      - `p -› prev`
- új listaelem létrehozása:
  - `p := new E2`

Egyszerű kétirányú láncolt lista - S2L



Ciklikus kétirányú láncolt lista - C2L



C2L műveletei

Precede

- egy listaelem beszúrása egy másik listaelem elé

Follow

- az első paraméterben kapott listaelem után beszúrjuk a másikat

Unlink

- a megadott listaelem kifűzése

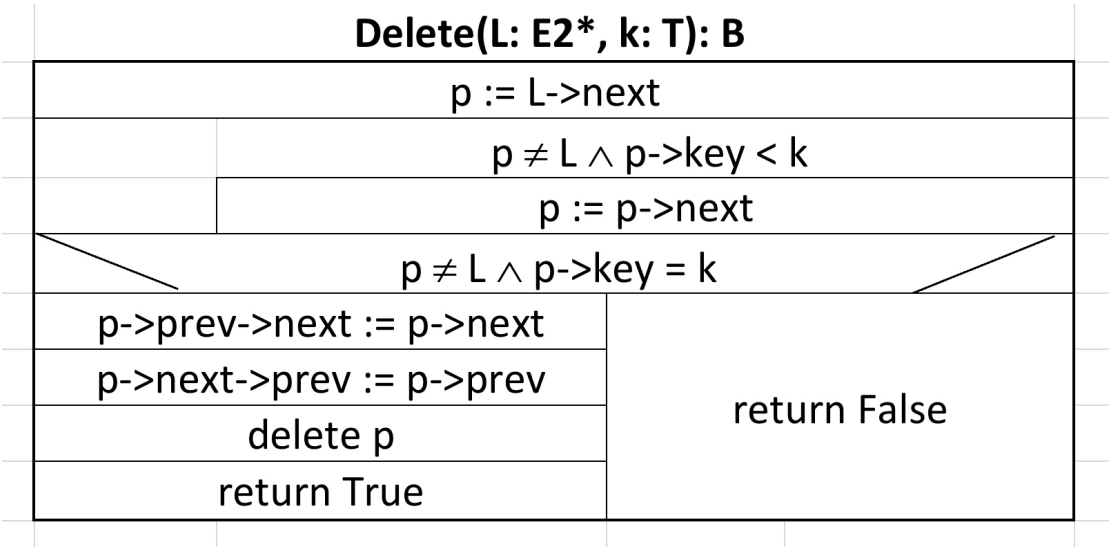
precede(q, r : E2*)
// $(*q)$ will precede $(*r)$
$p := r \rightarrow prev$
$q \rightarrow prev := p$; $q \rightarrow next := r$
$p \rightarrow next := r \rightarrow prev := q$

follow(p, q : E2*)
// $(*q)$ will follow $(*p)$
$r := p \rightarrow next$
$q \rightarrow prev := p$; $q \rightarrow next := r$
$p \rightarrow next := r \rightarrow prev := q$

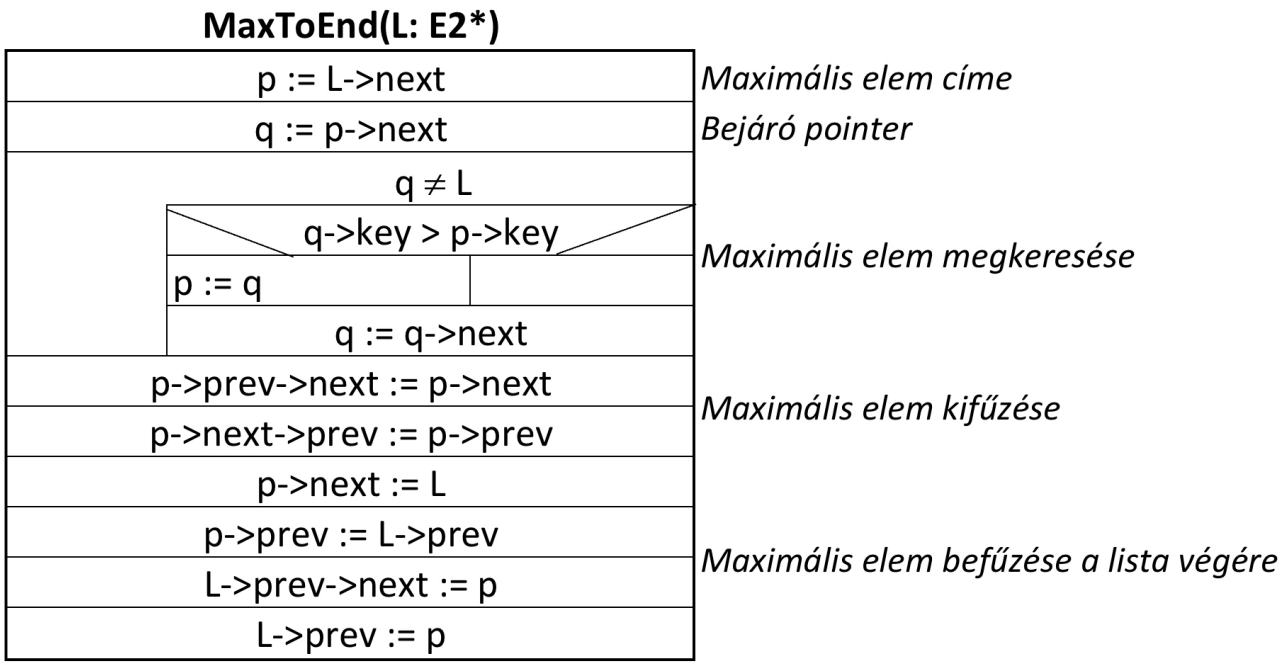
unlink(q : E2*)
// remove $(*q)$
$p := q \rightarrow prev$; $r := q \rightarrow next$
$p \rightarrow next := r$; $r \rightarrow prev := p$
$q \rightarrow prev := q \rightarrow next := q$

Törlés C2L listából

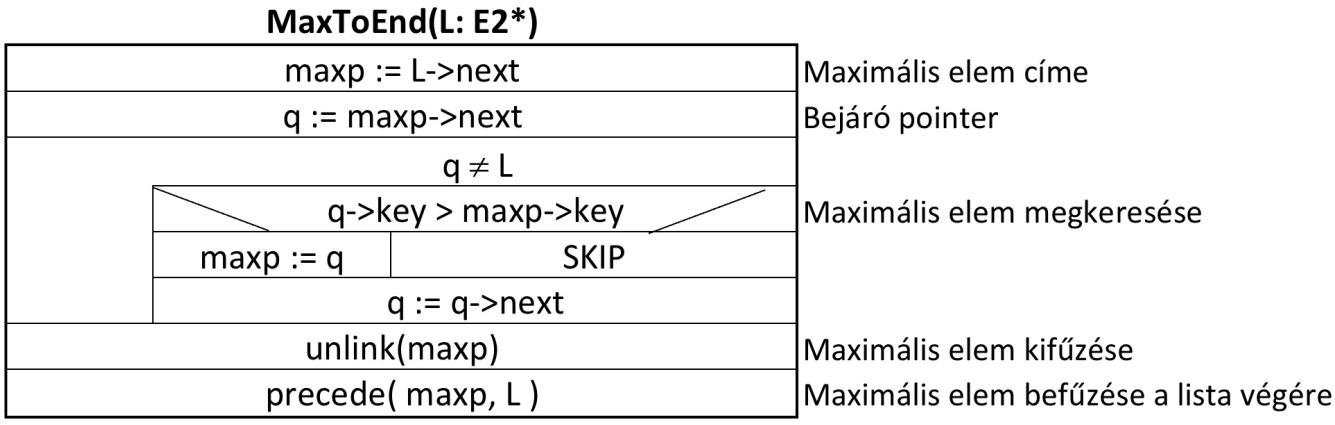
Szigorúan monoton növekvően rendezett C2L listából törli a paraméterül kapott kulcsú elemet, ha van ilyen. Logikai értékkel tér vssza.



Maximális elem a lista végére

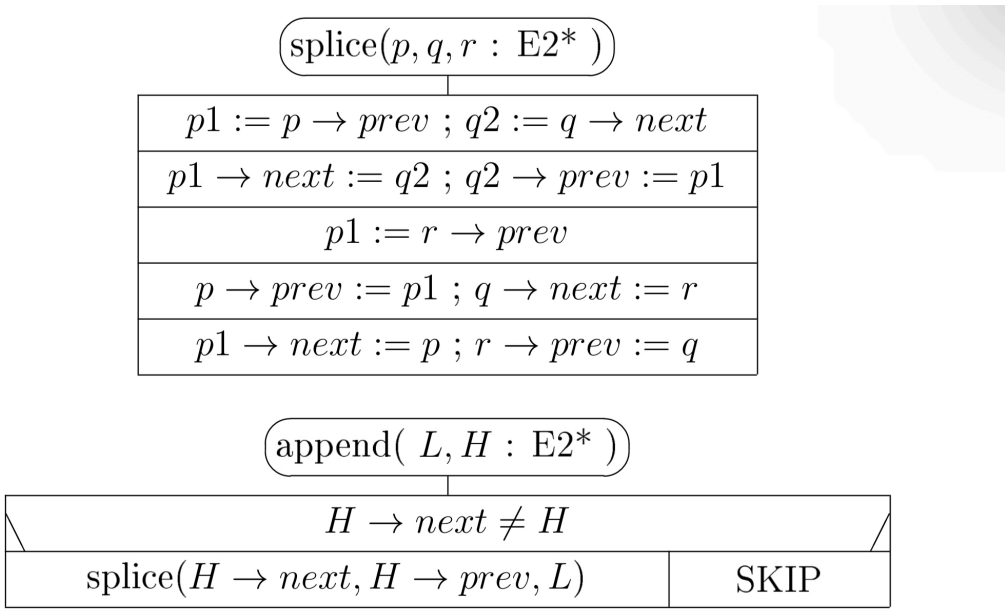


Az algoritmus a tanult listaműveletekkel:



Halmazok uniója

Adott két szigorúan monoton növekően rendezett C2L lista (halmazt ábrázolnak): L1, L2
L1-ben állítsuk elő a két halmaz unióját.



- összefuttatjuk a két listát
- az azonos elemeket töröljük L2-ből
- ha L1 végére értünk, de még L2-ben maradtak elemek, akkor azokat L1 végére fűzzük

union(L1, L2: E2*)

<table>
<tr><td colspan="4">p := L1->next ; q := L2->next</td></tr>
<tr><td rowspan="5"></td><td colspan="3">p ≠ L1 ∧ q ≠ L2</td></tr>
<tr><td>p->key < q->key</td><td>p->key = q->key</td><td>p->key > q->key</td></tr>
<tr><td rowspan="4">p := p->next</td><td>unlink(q)</td><td>unlink(q)</td></tr>
<tr><td>delete(q)</td><td rowspan="2">precede(q, p)</td></tr>
<tr><td>q := L2->next</td></tr>
<tr><td>p := p->next</td><td>q := L2->next</td></tr>
<tr><td colspan="4">append(L1, L2) // Ha L2-ben maradtak még elemek, ezeket átfűzzük L1 végére</td></tr>
</table>

- p-›key = q-›key : q című elemet kifűzzük L2-ből, majd töröljük
- p-›key > q-›key : a q című elemet átfűzzük L1-be a p című elem elé

Műveletigény

mT(n,m) = Θ(min(n,m))

MT(n,m) = Θ(n+m)

(L1 hossza: n, L2 hossza: m)

Sor adattípus

- FIFO (firstin first out) adatszerkezet

Sor megvalósítása egyirányú listával

Queue²
-first, last: E1* //*a sor első és utolsó elemére mutató pointerek* -size: N
+ Queue() + add(x: T) // *új elem hozzáadása a sor végére* + rem(): T // *a sor elején lévő elem eltávolítása* + first() : T // *a sor elején lévő elem lekérdezése* + length(): N + isEmpty(): B + ~Queue() + setEmpty()

Queue::isEmpty(): B

return size = 0

Queue::Queue()

first := last := new E1
size := 0

Queue::setEmpty()

first ≠ last	
	p := first
	first := first->next
	delete p
size:= 0	

Queue::length(): N

return size

Queue::~Queue()

first ≠ null	
	p := first
	first := first->next
	delete p

Queue::first(): T

size = 0	
Error	return first->key

Queue::rem(): T

size = 0	
Error	x := first->key
	s := first
	first := first->next
	delete s
	size := size - 1
	return x

Queue::add(x: T)

last->key := x
last->next := new E1
last := last->next
size := size + 1

Dadogós szöveg

Oldjuk meg két sor segítségével a következő feladatot:

Olvassunk be karakterenként egy szöveget (hossza nem ismert), és döntsük el, hogy „dadogós” –e.

Pl.: abcabc dadogós, abccbb nem dadogós

